Comparing O0 with O1, we can see that the differences in how the assembly is generated is within the printf helper function. Since O0 is pushing %rbp, and also moving %rbp to the stack, this means %rbp a frame pointer, which is basically like a central address where data will be retrieved and stored. O1 uses both registers. Both levels of optimization do not change LC or literal constants as they are strings which can not be optimized any further with either O1 and O0 level of optimization. In the main section of the assembly, O0 starts by pushing %rbp, while O1 pushes 8 different registers making initialization of registers in the stack slower for O1, but ultimately faster since we do not have to constantly be using the frame pointer for accessing variables outside and inside loops since every variable has its own register to point to. This means that from O0 to O1, while minimal, the optimization to level 1 show that level 0 generates assembly from the c program line by line, while O1 “looks ahead” to see how many times the loops iterate, and what variables and functions are most used so the assembler can decide which variables need which registers depending on the priority of the variables. Comparing O1 to O3, we see that printf is mostly similar to each other, only moving some commands around, like cloning %rcx to %rbx later compared to O1. A major difference between the code is also organization. O3 is significantly more organized than O1, as LC’s are stored below their respective printf .constprop. Adding to this, O3 also optimizes some printf calls in the c program by hardcoding a printf function for static printfs like printf("|"); since they can be called later for easier access when calling from main, making O3’s optimization of printf calls faster compared to O1. In main, Both O1 and O3 set up 8 registers for memory allocation but O1 sets up 56 bytes compared to O3 setting up 72 bytes to the %rsi register. Because O3 optimizes with const prop, this ultimately makes this version of the assembly faster since it can call static printf functions instead of having to make a general one. From this we can see how the 3 levels of optimization

changes the structure and organization of the assembly, where instead of reading the code top to bottom, it tries to shorten loops and calls in the assembly.